🌈 RS004 Week 2 — English Worksheet

Topic: Gradient Background (Apply) · **Course:** Platformer Game · **Time:** about 45 minutes

This week your flat sky becomes a **gradient** — a smooth blend from one colour at the top to another at the bottom. In this worksheet you will read gradient code, work out the blend maths on paper, and explain how the sky is drawn one line at a time.

> Keep these words handy: **gradient**, **palette**, **blend (interpolate)**, `draw.line`, **frame**.

1 · Predict 🔮

Read each snippet. Write what you think it does — just read it, no need to run it.

```
for y in range(HEIGHT):
    pygame.draw.line(screen, (100, 180, 255), (0, y), (WIDTH, y))
```

How many lines get drawn? What do they cover together?

```
t = y / HEIGHT
r = int(SKY_TOP[0] * (1 - t) + SKY_BOTTOM[0] * t)
```

At the very top (`y = 0`), what is `t`? Which colour wins — top or bottom?

```
SKY_TOP = (100, 180, 255)
SKY_BOTTOM = (200, 230, 255)
```

These sit at the top of the file with the other colours. Why keep all colours in one place?

2 · Blend Math 🧮

The blend formula is `int(A * (1 - t) + B * t)`. Fill in the table. `A = 100`, `B = 200`.

```
t = 0.0  → result = ___
t = 0.5  → result = ___
t = 1.0  → result = ___
```

In your own words: when `t` goes from 0 to 1, the colour moves from __ to ___.

3 · Spot the Bug 🐛

Each snippet is broken. Rewrite it so it works, then explain why the original was wrong.

Bug A — The sky should blend from top to bottom, but the whole screen comes out one flat colour.

```
for y in range(HEIGHT):
    t = 0
    r = int(SKY_TOP[0] * (1 - t) + SKY_BOTTOM[0] * t)
    g = int(SKY_TOP[1] * (1 - t) + SKY_BOTTOM[1] * t)
    b = int(SKY_TOP[2] * (1 - t) + SKY_BOTTOM[2] * t)
    pygame.draw.line(screen, (r, g, b), (0, y), (WIDTH, y))
```

Hint: `t` should *change* as `y` goes down. It is stuck at one value.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — The gradient should fill the screen, but only the very top line is coloured.

```
y = 0
t = y / HEIGHT
r = int(SKY_TOP[0] * (1 - t) + SKY_BOTTOM[0] * t)
pygame.draw.line(screen, (r, SKY_TOP[1], SKY_TOP[2]), (0, y), (WIDTH, y))
```

Hint: one line is drawn. How do we draw *every* row?

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Explain the Code 📖

Here is a working gradient background. Read it carefully, then answer the questions.

```
SKY_TOP = (100, 180, 255)
SKY_BOTTOM = (200, 230, 255)

def draw_gradient_background():
    for y in range(HEIGHT):
        t = y / HEIGHT
        r = int(SKY_TOP[0] * (1 - t) + SKY_BOTTOM[0] * t)
        g = int(SKY_TOP[1] * (1 - t) + SKY_BOTTOM[1] * t)
        b = int(SKY_TOP[2] * (1 - t) + SKY_BOTTOM[2] * t)
        pygame.draw.line(screen, (r, g, b), (0, y), (WIDTH, y))
```

What does the `for y in range(HEIGHT)` loop do, and how many lines does it draw?

The line `t = y / HEIGHT` — what value does `t` have at the top row, and what value near the bottom row?

Why does the code blend `r`, `g`, and `b` separately instead of just once?

What would happen to the sky if you swapped `SKY_TOP` and `SKY_BOTTOM`?

Why does `draw_gradient_background()` use the names `SKY_TOP` and `SKY_BOTTOM` instead of writing the numbers straight into the loop?

5 · Explain Your Lesson Code 🎥

In today's live lesson you wrote your own gradient sky. Now explain the code *you* wrote. Record a short video on your phone — you can show it running. Try to use these words: **gradient**, **palette**, **blend**, **line**, **loop**.

> 1. Show your gradient sky running and say which two palette colours you used (top and bottom).
> 2. Point at your `for` loop and explain how it draws the sky line by line.
> 3. Read your blend line out loud and explain what `t` does as `y` goes down.
> 4. Show one colour you changed and say what changed on screen.

Write what you will say in your video. Plan it here before you record.

Submit ✅

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.